

ECE 374 B: Algorithms and Models of Computation, Summer 2026

Midterm 2 – Solutions

- **You will have 75 minutes (1.25 hours) to solve all the problems. Most have multiple parts.** Don't spend too much time on questions you don't understand and focus on answering as much as you can! Make sure to check both sides of all the pages and make sure you answered everything. **Time is a factor!** Budget yours wisely.
 - No resources are allowed for use during the exam except a multi-page cheatsheet and scratch paper on the back of the exam. ***Do not tear out the cheatsheet or the scratch paper!*** It messes with the auto-scanner.
 - You should write your answers *completely* in the space given for the question. We will not grade parts of any answer written outside of the designated space.
 - Please *use a dark-colored pen* unless you are *absolutely* sure your pencil writing is forceful enough to be legible when scanned. We will take off points if we have difficulty reading the uploaded document.
 - Incorrect algorithms will receive a score of 0, but slower than necessary but correct algorithms will *always* receive some points, even brute force ones. Thus, *you should prioritize the correctness of your submitted algorithms over speed*; you will receive more points that way. On the other hand, submit the fastest algorithms that you know are correct; faster algorithms will receive more points.
 - Any recursive backtracking algorithm or dynamic programming algorithm given without an *English* description of the recursive function (i.e., a description of the output of the function *in terms of their inputs*) will receive a score of 0.
 - Any greedy algorithm or a modification of a standard graph algorithm given without a proof of correctness will receive a score of 0.
 - For problems with a graph given as input, you may assume the graph is simple (i.e., it has no self-loops or parallel edges).
 - Only algorithms referenced in the cheat sheet may be referred to as a "black box". You may not simply refer to a prior lab/homework for the solution and must give the full answer.
 - Unless explicitly mentioned, **a runtime analysis is required for each given algorithm.**
 - ***Don't cheat.*** If we catch you, you will get an F in the course.
 - ***Good luck!***
-

Name: _____

NetID: _____

1 Bounding functions - 10 points

For each of the following recurrences, provide a **tight asymptotic upper bound**. Circle your final answer. No partial credit.

(a)

$$A(n) = A(n/4) + \sqrt{n} + O(n)$$

Solution:

$$A(n) = A(n/4) + \sqrt{n} + O(n)$$

$O(n)$ dominates work done at each level (since we trying to find upper bound)

$$A(n) = A(n/4) + O(n).$$

The recursion tree has height $\log_4 n$, since the input size is divided by 4 at each level until it reaches 1.

Expanding the recurrence,

$$\begin{aligned} A(n) &= O(n) + O(n/4) + O(n/4^2) + \dots \\ &= O\left(n \sum_{i=0}^{\log_4 n} \left(\frac{1}{4}\right)^i\right) = \sum_{i=0}^{\infty} \left(\frac{1}{4}\right)^i = O\left(\frac{4}{3}n\right) \end{aligned}$$

Therefore,

$$A(n) = O(n).$$

■

(b)

$$B(n) = B(n/2) + 2B(n/4) + O(n)$$

Solution:

$$B(n) = B(n/2) + 2B(n/4) + O(n)$$

At each level, the total subproblem size is

$$\frac{n}{2} + 2\left(\frac{n}{4}\right) = n,$$

so the work per level is $O(n)$.

The recursion tree has height $O(\log n)$, since the longest branch reduces the input size by a factor of 2.

Since work done at each level is constant,

$$B(n) = O(n \log n).$$



2 Short Answer I - Algos 20 points

Answer the following questions. You may **briefly** (no more than 2 sentences) justify your answers, but a complete proof is not required. Partial credit is limited.

- (a) Provide the recurrence for the longest **common** subsequence (you are given two sequences, A & B and you need to return the length of the longest subsequence that occurs in both sequences). The smallest edit distance algorithm is in the notes but remember, you have to modify it slightly to get the longest common subsequence algorithm.

Solution:

$$\text{LCS}(i, j) = \begin{cases} 0, & \text{if } i > n \text{ or } j > m, \\ 1 + \text{LCS}(i + 1, j + 1), & \text{if } A[i] = B[j], \\ \max\{\text{LCS}(i + 1, j), \text{LCS}(i, j + 1)\}, & \text{if } A[i] \neq B[j]. \end{cases}$$

This recurrence considers the two cases:

- If the current characters match, we include this character in the LCS.
- Otherwise, we skip one character from either sequence and take the better option.

Alternative (Black-Box) Solution:

The length of the longest common subsequence can also be computed using the edit distance algorithm discussed in class. When only insertions and deletions are allowed,

$$\text{LCS length} = \frac{n + m - \text{EditDistance}(A, B)}{2}.$$

■

- (b) You are given two character sequences $A[0 \dots n - 1]$, $B[0 \dots m - 1]$. Assuming no other constraints on A, **under what condition (for B) is the alignment between these two sequences equal to n?** Assume mismatch cost (α) and insertion/deletion (δ) costs are both equal to 1. (There are multiple correct answers, give one).

Solution: The alignment cost is the minimum cost required to transform A into B, where matches cost 0, mismatches cost 1, and insertions/deletions cost 1.

Condition 1: $B = \epsilon$.

All n characters of A must be deleted, so

$$\text{cost}(A, B) = n.$$

Condition 2: $m = n$ and A and B have no characters in common.

Every aligned pair is a mismatch, so

$$\text{cost}(A, B) = n.$$

Condition 3: $m = 2n$ and A is a subsequence of B .

The n characters of A match at zero cost, while the remaining n characters are inserted. Hence,

$$\text{cost}(A, B) = n.$$

■

- (c) Recall in lecture/discussion we discussed the median of median (linear time selection) algorithm. The algorithm we discussed breaks a array into lists of size five. What if we break the array into lists of size 7. **What is the recurrence and asymptotic running time** for the this modified version of linear time selection?

Solution: For groups of size 7, each group median is the 4th-smallest element. At least half of the group medians are at least the pivot, so at least

$$4 \left(\frac{1}{2} \right) \left(\frac{n}{7} \right) = \frac{2n}{7}$$

elements are guaranteed to be discarded. Thus, the larger recursive partition has size at most

$$n - \frac{2n}{7} = \frac{5n}{7}.$$

Therefore,

$$T(n) \leq T(n/7) + T(5n/7) + O(n).$$

Since

$$\frac{1}{7} + \frac{5}{7} = \frac{6}{7} < 1,$$

the total work decreases geometrically, giving

$$T(n) = O(n).$$

■

- (d) Recall in lecture/discussion we discussed the MergeSort algorithm. The algorithm we discussed breaks an array into two parts. **What if we break the array into three parts. What is the recurrence and asymptotic running time** for the this modified version of the merge-sort algorithm?

Solution: Splitting the array into three equal parts gives

$$T(n) = 3T\left(\frac{n}{3}\right) + O(n).$$

At each level of the recursion tree, the total merging cost is constant:

$$O(n).$$

The recursion has depth

$$\log_3 n.$$

Therefore,

$$T(n) = O(n) \cdot O(\log_3 n) = O(n \log n).$$



3 Short Answer II - Graph Algos 15 points

- (a) Given a directed graph G , you perform Kosaraju's algorithm to get the meta-graph of strongly connected components of G . But then you realize the meta-graph and original graph are exactly the same ($G = \text{SCC}(G)$). What type of graph is G ?

Solution: Kosaraju's algorithm creates a meta-graph by replacing each strongly connected component (SCC) of G with a single vertex. An edge is added between two meta-vertices if there is an edge between their corresponding SCCs in the original graph.

Given that

$$G = \text{SCC}(G),$$

forming the meta-graph does not change the graph. Therefore, no group of multiple vertices can belong to the same SCC; otherwise, those vertices would be contracted into one vertex.

Thus, every SCC contains exactly one vertex. Since any directed cycle would place all vertices on that cycle in the same SCC, G cannot contain any directed cycles.

Therefore,

G is a directed acyclic graph (DAG).

■

- (b) Suppose we want to know the shortest distance between all pairs of nodes. The natural thing to do would be to use the Floyd Warshall algorithm but in this case, you realize it is better to simply run Dijkstra's algorithm for every possible start vertex in the graph. **Under what condition is it better to run Dijkstra's algorithm $|V|$ times than simply run the Floyd-Warshall algorithm once?**

Solution: Running Dijkstra's algorithm from every vertex using a Fibonacci heap takes

$$O(VE + V^2 \log V),$$

while Floyd-Warshall takes

$$O(V^3).$$

If the graph has no negative-weight edges and

$$E = o(V^2),$$

then repeated Dijkstra is asymptotically faster. For a fully connected graph, $E = \Theta(V^2)$, so

$$O(VE + V^2 \log V) = O(V^3),$$

which is the same asymptotic bound as Floyd–Warshall.

The condition is that the graph must not have negative edge weights ■

- (c) You are given a graph G and vertex s and need to provide the shortest path between s and t . Looking at the G you realize that all the edge weights are equal to 1. Briefly describe an algorithm that can give you the shortest path.

Solution: Since all edge weights are equal to 1, the shortest path is the path containing the fewest edges. Run Breadth-First Search (BFS) starting from s . BFS explores vertices in increasing order of their distance from s , so the first time t is reached, a shortest path to t has been found.

Store the parent of each vertex when it is first discovered. Then, reconstruct the shortest path by following the parent pointers from t back to s . ■

4 Simple algorithms - 10 points

You are given a **sorted** array ($A[1..n]$) that may contain duplicate values. Describe an algorithm that finds whether array A contains more than $\frac{n}{4}$ copies of any particular value. You don't know what the value is. The algorithm should run in linear time but extraneous steps will result in a point deduction (even if the asymptotic bound is the same).

Solution: Since this is a sorted array, if a value appears for more than $\frac{n}{4}$ times, it must appear in the following candidate positions: $\{A[\frac{n}{4}], A[\frac{n}{2}], A[\frac{3n}{4}]\}$, so to find whether array A contains more than $\frac{n}{4}$ copies of any particular value, we just need to count the number of times values in these three candidate positions appears.

```
candidates = [A[ $\frac{n}{4}$ ], A[ $\frac{n}{2}$ ], A[ $\frac{3n}{4}$ ]]  
  
for x in candidates  
    count = 0  
  
    for i=1 to n  
        if A[i] == x  
            count += 1  
  
    if count > n/4  
        return True  
  
return False
```

■

5 Dynamic programming - 15 points

You are given the sequence ($A[1..n]$) and you want to find the longest palidrome subsequence (the longest suqbssequence that is the same forward and back).

Recurrence and short English description(in terms of the parameters):

Solution: Let $D[i][j]$ be the length of the longest palindromic subsequence in the subarray $A[i..j]$.

Base cases:

$D[i][j] = 0$ if $i > j$

$D[i][i] = 1$

Recurrence:

if $A[i] == A[j]$, then $D[i][j] = D[i+1][j-1] + 2$

if $A[i] != A[j]$, then $\max(D[i+1][j], D[i][j-1])$

The final answer is $DP[1][n]$



Memoization and Evaluation Order

Solution: Store all values $DP[i][j]$ in a 2D table of size n by n .

Initialize $DP[j][j] = 1$ for each column j .

We evaluate the table column by column from left to right. That is, we increase the right endpoint j from 1 to n . For each fixed j , we compute $DP[i][j]$ for $i = j, j-1, \dots, 1$.

This order is valid because $DP[i][j]$ depends on $DP[i+1][j]$, $DP[i][j-1]$, and $DP[i+1][j-1]$. When computing column j from bottom to top, $DP[i+1][j]$ has already been computed in the same column, and $DP[i][j-1]$ and $DP[i+1][j-1]$ were computed in the previous column.



6 Graphing recursions - 15 points

Below, you are given Dijkstra's algorithm seen in lecture which outputs the *cost* of the shortest path to any vertex given some starting vertex.

Algorithm 1 Dijkstra's Algorithm

```
1:  $Q \leftarrow \text{makePQ}()$ 
2:  $\text{insert}(Q, (s, 0))$ 
3: for all  $u \in V, u \neq s$  do
4:    $\text{insert}(Q, (u, \infty))$ 
5: end for

6:  $X \leftarrow \emptyset$ 
7: for  $i = 1$  to  $|V|$  do
8:    $(v, \text{DIST}(s, v)) \leftarrow \text{extractMin}(Q)$ 
9:    $X \leftarrow X \cup \{v\}$ 
10:  for all  $u \in \text{Adj}(v)$  do
11:     $\text{decreaseKey}(Q, (u, \min\{\text{DIST}(s, u), \text{DIST}(s, v) + \ell(v, u)\}))$ 
12:  end for
13: end for

14: return  $\text{DIST}(s, :)$ 
```

Modify the algorithm so that it outputs the shortest path to any vertex v given a starting vertex s (you can just say “add <code> after line #” etc. What is the running time of your new algorithm?

Solution: Suppose vertex v^* is the target vertex, then we can save path to any vertex in Dijkstra:

Algorithm 2 Output shortest path

```
1:  $Q \leftarrow \text{MakePQ}$ 
2:  $\text{Insert}(Q, (s, 0))$ 
3:  $\text{parent}(s) \leftarrow \text{NULL}$ 
4: for all  $u \in V, u \neq s$  do
5:    $\text{Insert}(Q, (u, \infty))$ 
6:    $\text{parent}(u) \leftarrow \text{NULL}$ 
7: end for
8:  $X \leftarrow \emptyset$ 
9: for  $i = 1$  to  $|V|$  do
10:   $(v, \text{DIST}(s, v)) \leftarrow \text{ExtractMin}(Q)$ 
11:   $X \leftarrow X \cup \{v\}$ 
12:  if  $v = v^*$  then
13:    return  $\text{RecoverPath}(\text{parent}, v^*)$ 
14:  end if
15:  for all  $u \in \text{Adj}(v)$  do
16:    if  $\text{DIST}(s, v) + \ell(v, u) < \text{DIST}(s, u)$  then
17:       $\text{DecreaseKey}(Q, (u, \text{DIST}(s, v) + \ell(v, u)))$ 
18:       $\text{parent}(u) \leftarrow v$ 
19:    end if
20:  end for
21: end for
22: return No-Path
```

$|V|$ is number of vertices, $|E|$ is number of edges, then the time complexity is $O(|E| \log |V|)$ ■

7 Graph Algos - 15 points

Let $G = (V, E)$ be a directed graph with edge length $\ell : E \rightarrow \mathbb{R}^+$. A subset of the edges $E' \subseteq E$ are considered risky. Describe an algorithm that given $G = (V, E)$, the edge lengths ℓ , the risky subset E' and a node s , find for each node $v \in V$ the shortest path distance from s to v among all paths that contain **at most 1 risky edge**.

Solution: We can construct a two-layer graph and run Dijkstra's algorithm.

For each vertex $v \in V$, create two states:

- (1) $(v, 0)$: no risky edges has been used,
- (2) $(v, 1)$: exactly one risky edge has been used.

For every non-risky edge $(u, v) \in E \setminus E'$, add $(u, 0) \rightarrow (v, 0)$ and $(u, 1) \rightarrow (v, 1)$, both with length $\ell(u, v)$

Similarly, for every risky edge $(u, v) \in E'$, add $(u, 0) \rightarrow (v, 1)$ with length $\ell(u, v)$.

We don't add any edge from a $(u, 1)$ state using risky edge, since that would allow more than one risky edge. Then run Dijkstra's algorithm from the source state $(s, 0)$. For each original vertex v , the desired answer is $d(v) = \min\{d(v, 0), d(v, 1)\}$. ■

ECE 374 B Algorithms: Cheatsheet

1 Recursion

Simple recursion

- **Reduction:** solve one problem using the solution to another.
- **Recursion:** a special case of reduction - reduce problem to a *smaller* instance of *itself* (self-reduction).

Definitions

- Problem instance of size n is reduced to *one or more* instances of size $n - 1$ or less.
- For termination, problem instances of small size are solved by some other method as *base cases*

Arguably the most famous example of recursion. The goal is to move n disks one at a time from the first peg to the last peg.

Pseudocode: Tower of Hanoi

```
Hanoi(n, src, dest, tmp):  
  if (n > 0) then  
    Hanoi(n - 1, src, tmp, dest)  
    Move disk n from src to dest  
    Hanoi(n - 1, tmp, dest, src)
```

Tower of Hanoi

Divide and conquer

Divide and conquer is an algorithm paradigm involving the decomposition of a problem into the same subproblem, solving them separately and combining their results to get a solution for the original problem.

	Algorithm	Runtime	Space
Sorting algorithms	Mergesort	$O(n \log n)$	$O(n \log n)$ $O(n)$ (if optimized)
	Quicksort	$O(n^2)$ $O(n \log n)$ if using MoM	$O(n)$

We can divide and conquer multiplication like so:

$$bc = 10^n b_L c_L + 10^{n/2} (b_L c_R + b_R c_L) + b_R c_R.$$

We can rewrite the equation as:

$$bc = b(x)c(x) = (b_L x + b_R)(c_L x + c_R) = (b_L c_L)x^2 + ((b_L + b_R)(c_L + c_R) - b_L c_L - b_R c_R)x + b_R c_R,$$

Karatsuba's algorithm

Its running time is $O(n^{\log_2 3}) = O(n^{1.585})$.

Recurrences

Suppose you have a recurrence of the form $T(n) = rT(n/c) + f(n)$.

The *master theorem* gives a good asymptotic estimate of the recurrence. If the work at each level is:

Decreasing: $r f(n/c) = \kappa f(n)$ where $\kappa < 1$ $T(n) = O(f(n))$
Equal: $r f(n/c) = f(n)$ $T(n) = O(f(n) \cdot \log_c n)$
Increasing: $r f(n/c) = K f(n)$ where $K > 1$ $T(n) = O(n^{\log_c r})$

Some useful identities:

- Sum of integers: $\sum_{k=1}^n k = \frac{n(n+1)}{2}$
- Geometric series closed-form formula: $\sum_{k=0}^n ar^k = a \frac{1-r^{n+1}}{1-r}$
- Logarithmic identities: $\log(ab) = \log a + \log b$, $\log(a/b) = \log a - \log b$, $a^{\log_c b} = b^{\log_c a}$ ($a, b, c > 1$), $\log_a b = \log_c b / \log_c a$.

Backtracking

Backtracking is the algorithm paradigm involving guessing the solution to a single step in some multi-step process and recursing backwards if it doesn't lead to a solution. For instance, consider the longest increasing subsequence (LIS) problem. You can either check all possible subsequences:

Pseudocode: LIS - Naive enumeration

```
algLISNaive(A[1..n]):  
  maxmax = 0  
  for each subsequence B of A do  
    if B is increasing and |B| > max then  
      max = |B|  
  return max
```

On the other hand, we don't need to generate every subsequence; we only need to generate the subsequences that are increasing:

Pseudocode: LIS - Backtracking

```
LIS_smaller(A[1..n], x):  
  if n = 0 then return 0  
  max = LIS_smaller(A[1..n-1], x)  
  if A[n] < x then  
    max = max {max, 1 + LIS_smaller(A[1..(n-1)], A[n])}  
  return max
```

Linear time selection

The *median of medians* (MoM) algorithms give a element that is larger than $\frac{3}{10}$'s and smaller than $\frac{7}{10}$'s of the array elements. This is used in the linear time selection algorithm to find element of rank k .

Pseudocode: Quickselect with median of medians

```
Median-of-medians(A, i):  
  sublists = [A[j:5] for j ← 0, 5, ..., len(A)]  
  medians = [sorted(sublist)[len(sublist)/2] for sublist in sublists]  
  
  // Base case  
  if len(A) ≤ 5 return sorted(a)[i]  
  
  // Find median of medians  
  if len(medians) ≤ 5  
    pivot = sorted(medians)[len(medians)/2]  
  else  
    pivot = Median-of-medians(medians, len/2)  
  
  // Partitioning step  
  low = l | for j ∈ A if j < pivot  
  high = l | for j ∈ A if j > pivot  
  
  k = len(low)  
  if i < k  
    return Median-of-medians(low, i)  
  else if i > k  
    return Median-of-medians(high, i-k-1)  
  else  
    return pivot
```

Dynamic programming

Dynamic programming (DP) is the algorithm paradigm involving the computation of a recursive backtracking algorithm iteratively to avoid the recomputation of any particular subproblem.

Longest increasing subsequence

The longest increasing subsequence problem asks for the length of a longest increasing subsequence in a unordered sequence, where the sequence is assumed to be given as an array. The recurrence can be written as:

$$LIS(i, j) = \begin{cases} 0 & \text{if } i = 0 \\ LIS(i-1, j) & \text{if } A[i] \geq A[j] \\ \max \begin{cases} LIS(i-1, j) \\ 1 + LIS(i-1, i) \end{cases} & \text{else} \end{cases}$$

Pseudocode: LIS - DP

LIS-Iterative($A[1..n]$):

$A[n+1] = \infty$

for $j \leftarrow 0$ **to** n

if $A[i] \leq A[j]$ **then** $LIS[0][j] = 1$

for $i \leftarrow 1$ **to** $n-1$ **do**

for $j \leftarrow i$ **to** $n-1$ **do**

if $A[i] \geq A[j]$

$LIS[i, j] = LIS[i-1, j]$

else

$LIS[i, j] = \max \{ LIS[i-1, j], 1 + LIS[i-1, i] \}$

return $LIS[n, n+1]$

Edit distance

The edit distance problem asks how many edits we need to make to a sequence for it to become another one. The recurrence is given as:

$$\text{Opt}(i, j) = \min \begin{cases} \alpha_{x_i y_j} + \text{Opt}(i-1, j-1), \\ \delta + \text{Opt}(i-1, j), \\ \delta + \text{Opt}(i, j-1) \end{cases}$$

Base cases: $\text{Opt}(i, 0) = \delta \cdot i$ and $\text{Opt}(0, j) = \delta \cdot j$

Pseudocode: Edit distance - DP

EDIST($A[1..m], B[1..n]$)

for $i \leftarrow 1$ **to** m **do** $M[i, 0] = i\delta$

for $j \leftarrow 1$ **to** n **do** $M[0, j] = j\delta$

for $i = 1$ **to** m **do**

for $j = 1$ **to** n **do**

$$M[i][j] = \min \begin{cases} \text{COST}[A[i]][B[j]] \\ \quad + M[i-1][j-1], \\ \delta + M[i-1][j], \\ \delta + M[i][j-1] \end{cases}$$

2 Graph algorithms

Graph basics

A graph is defined by a tuple $G = (V, E)$ and we typically define $n = |V|$ and $m = |E|$. We define (u, v) as the edge from u to v . Graphs can be represented as **adjacency lists**, or **adjacency matrices** though the former is more commonly used.

- **path**: sequence of *distinct* vertices $v_1, v_2, \dots, v_k$ such that $v_i v_{i+1} \in E$ for $1 \leq i \leq k-1$. The length of the path is $k-1$ (the number of edges in the path).
Note: a single vertex u is a path of length 0.
- **cycle**: sequence of *distinct* vertices $v_1, v_2, \dots, v_k$ such that $(v_i, v_{i+1}) \in E$ for $1 \leq i \leq k-1$ and $(v_k, v_1) \in E$. A single vertex is not a cycle according to this definition.
Caveat: Sometimes people use the term cycle to also allow vertices to be repeated; we will use the term *tour*.
- A vertex u is *connected* to v if there is a path from u to v .
- The *connected component* of u , $\text{con}(u)$, is the set of all vertices connected to u .
- A vertex u can *reach* v if there is a path from u to v . Alternatively v can be reached from u . Let $\text{rch}(u)$ be the set of all vertices reachable from u .

Directed acyclic graphs

Directed acyclic graphs (dags) have an intrinsic ordering of the vertices that enables dynamic programming algorithms to be used on them.
A *topological ordering* of a dag $G = (V, E)$ is an ordering $<$ on V such that if $(u, v) \in E$ then $u < v$.

Pseudocode: Kahn's algorithm

```
Kahn( $G(V, E), u$ ):
  toposort  $\leftarrow$  empty list
  for  $v \in V$ :
     $in(v) \leftarrow |\{u \mid u \rightarrow v \in E\}|$ 
  while  $v \in V$  that has  $in(v) = 0$ :
    Add  $v$  to end of toposort
    Remove  $v$  from  $V$ 
    for  $v$  in  $u \rightarrow v \in E$ :
       $in(v) \leftarrow in(v) - 1$ 
  return toposort
```

Running time: $O(n + m)$

- A dag may have multiple topological sorts.
- A topological sort can be computed by DFS, in particular by listing the vertices in decreasing post-visit order.

DFS and BFS

Pseudocode: Explore (DFS/BFS)

```
Explore( $G, u$ ):
  for  $i \leftarrow 1$  to  $n$ :
    Visited[ $i$ ]  $\leftarrow$  False
  Add  $u$  to ToExplore and to  $S$ 
  Visited[ $u$ ]  $\leftarrow$  True
  Make tree  $T$  with root as  $u$ 
  while  $B$  is non-empty do
    Remove node  $x$  from  $B$ 
    for each edge  $(x, y)$  in  $Adj(x)$  do
      if Visited[ $y$ ] = False
        Visited[ $y$ ]  $\leftarrow$  True
        Add  $y$  to  $B, S, T$  (with  $x$  as parent)
```

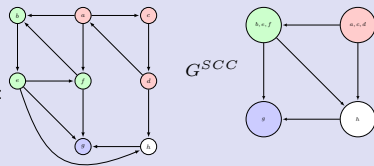
Note:

- If B is a queue, *Explore* becomes BFS.
- If B is a stack, *Explore* becomes DFS.

Pre and post numbering aids in analyzing the graph structure. By looking at the numbering we can tell if a edge (u, v) is a:

Pre/post numbering

- Forward edge: $pre(u) < pre(v) < post(v) < post(u)$
- Backward edge: $pre(v) < pre(u) < post(u) < post(v)$
- Cross edge: $pre(u) < post(u) < pre(v) < post(v)$



Strongly connected components

- Given G , u is *strongly connected* to v if $v \in rch(u)$ and $u \in rch(v)$.
- A *maximal* group of G : vertices that are all strongly connected to one another is called a strong component.

Pseudocode: Metagraph - linear time

```
Metagraph( $G(V, E)$ ):
  Compute  $rev(G)$  by brute force
  ordering  $\leftarrow$  reverse postordering of  $V$  in  $rev(G)$ 
  by DFS( $rev(G), s$ ) for any vertex  $s$ 
  Mark all nodes as unvisited
  for each  $u$  in ordering do
    if  $u$  is not visited and  $u \in V$  then
       $S_u \leftarrow$  nodes reachable by  $u$  by DFS( $G, u$ )
      Output  $S_u$  as a strong connected component
       $G(V, E) \leftarrow G - S_u$ 
```

Shortest paths

Dijkstra's algorithm:

Find minimum distance from vertex s to **all** other vertices in graphs *without* negative weight edges.

Pseudocode: Dijkstra

```
for  $v \in V$  do
   $d(v) \leftarrow \infty$ 
 $X \leftarrow \emptyset$ 
 $d(s, s) \leftarrow 0$ 
for  $i \leftarrow 1$  to  $n$  do
   $v \leftarrow \arg \min_{u \in V - X} d(u)$ 
   $X = X \cup \{v\}$ 
  for  $u$  in  $Adj(v)$  do
     $d(u) \leftarrow \min\{d(u), d(v) + \ell(v, u)\}$ 
return  $d$ 
```

Running time: $O(m + n \log n)$ (if using a Fibonacci heap as the priority queue)

Bellman-Ford algorithm:

Find minimum distance from vertex s to **all** other vertices in graphs *without* negative cycles. It is a DP algorithm with the following recurrence:

$$d(v, k) = \begin{cases} 0 & \text{if } v = s \text{ and } k = 0 \\ \infty & \text{if } v \neq s \text{ and } k = 0 \\ \min \begin{cases} \min_{u \in E} \{d(u, k-1) + \ell(u, v)\} \\ d(v, k-1) \end{cases} & \text{else} \end{cases}$$

Base cases: $d(s, 0) = 0$ and $d(v, 0) = \infty$ for all $v \neq s$.

Pseudocode: Bellman-Ford

```
for each  $v \in V$  do
   $d(v) \leftarrow \infty$ 
 $d(s) \leftarrow 0$ 
for  $k \leftarrow 1$  to  $n - 1$  do
  for each  $v \in V$  do
    for each edge  $(u, v) \in in(v)$  do
       $d(v) \leftarrow \min\{d(v), d(u) + \ell(u, v)\}$ 
return  $d$ 
```

Running time: $O(nm)$

Floyd-Warshall algorithm:

Find minimum distance from *every* vertex to *every* vertex in a graph *without* negative cycles. It is a DP algorithm with the following recurrence:

$$d(i, j, k) = \begin{cases} 0 & \text{if } i = j \\ \infty & \text{if } (i, j) \notin E \text{ and } k = 0 \\ \min \begin{cases} d(i, j, k-1) \\ d(i, k, k-1) + d(k, j, k-1) \end{cases} & \text{else} \end{cases}$$

Then $d(i, j, n-1)$ will give the shortest-path distance from i to j .

Pseudocode: Floyd-Warshall

```
Metagraph( $G(V, E)$ ):
  for  $i \in V$  do
    for  $j \in V$  do
       $d(i, j, 0) \leftarrow \ell(i, j)$ 
      (*  $\ell(i, j) \leftarrow \infty$  if  $(i, j) \notin E, 0$  if  $i = j$  *)
  for  $k \leftarrow 0$  to  $n - 1$  do
    for  $i \in V$  do
      for  $j \in V$  do
         $d(i, j, k) \leftarrow \min \begin{cases} d(i, j, k-1), \\ d(i, k, k-1) + d(k, j, k-1) \end{cases}$ 
  for  $v \in V$  do
    if  $d(i, i, n-1) < 0$  then
      return "negative cycle in  $G$ "
  return  $d(\cdot, \cdot, n-1)$ 
```

Running time: $\Theta(n^3)$